

Data Structure

CO4 - AT1

Comparative Analysis

S. THANISHKA

192521457

B. Tech IT

CSA0307

SHOTA

① Compare Heap Sort and Quick Sort in terms of.

a) Worst-case Guarantees:

Heap Sort:

=> Heap Sort has a guaranteed worst-case time complexity of $O(n \log n)$

=> It builds a heap and repeatedly removes the maximum / minimum element

=> Even if the input is already sorted or arranged in an unfavorable order, its performance remains $O(n \log n)$.

Quick Sort:

=> Quick Sort has an average-case time

complexity of $O(n \log n)$

=> However, its worst-case time complexity is $O(n^2)$

b) Practical Performance

Compare Radix Sort & Counting Sort

Criteria	Radix Sort	Counting Sort
Working Mechanism	Sorts elements digit by digit	Sorts elements by counting occurrences of each value
Basic Idea	Processes units, tens, hundreds, etc.	Creates a count array for input values
Range of input values	Suitable for integers with a large range but limited number of digits.	Best when the input values have a small range
Time complexity	$O(d \times (n+k))$	$O(n+k)$
Space complexity	$O(n+k)$	$O(n+k)$
Efficiency	Efficient when the number of digits d is small	Very efficient when k is small
Example	Sorting 170, 45, 75, 90 digit by digit	Sorting marks from 0 to 100
Advantage	Handles large integer values efficiently	Simple and very fast for small ranges.

Heap Sort:

$\Rightarrow$ Heap sort performs consistently well for different input arrangements.

$\Rightarrow$ However, it generally has more memory access operations and poorer cache locality.

$\Rightarrow \therefore$ It is often slower.

Quick Sort:

$\Rightarrow$ It is usually very fast in practical situations.

$\Rightarrow$ It works on smaller subarrays after partitioning and generally has good cache performance.

$\Rightarrow$ It performs close to $O(n \log n)$.

c) Use Cases:

$\Rightarrow$ Heaps sort is useful when guaranteed $O(n \log n)$ performance is required.

$\Rightarrow$ Worst-case performance is important

$\Rightarrow$ Quick Sort is useful when fast-average case performance is the main requirement

$\Rightarrow$ Large arrays need to be sorted efficiently